Universidad Rafael Landívar
Facultad de Ingeniería
Ingeniería en Informática y Sistemas
Área de Informática
Estructura de Datos II

PROYECTO DE LABORATORIO No. 2

Manual técnico

c

Integrante	Carné	Estructuras a su cargo
Integrante 1/ Hervert Alfredo Sales Gómez	1635825	Tabla Hash, Trie, Union-Find, Lista enlazada, carga de archivos, interfaz gráfica
Integrante 2/ Dan Mahli Soberanis Galindo	1658925	Grafo (BFS, DFS, Dijkstra) y MinHeap, carga de archivos, interfaz gráfica

Contenido

1. Introducción 2
1.1 Objetivo 2
1.2 Escenario elegido 2
1.3 Datos y alcance 2
1.4 Tecnologías utilizadas 2
2. Arquitectura del sistema 3
2.1 Organización del proyecto 3
2.2 Responsables por estructura 3
2.3 Diagrama de clases 4
2.4 Integración de las estructuras 4
2.5 Diagrama de flujo 6
3. Implementación de las estructuras 8
3.1 Lista enlazada (soporte) 8
3.2 Tabla Hash 8
3.3 Trie (árbol de prefijos) 9
3.4 Union-Find (conjuntos disjuntos) 9
3.5 Grafo, recorridos y Dijkstra 10
3.6 Carga y guardado de archivos 11
3.7 Interfaz gráfica y mapa 11
4. Análisis de complejidad (Big-O) 12
4.1 Tabla Hash 12
4.2 Trie 12
4.3 Union-Find 12
4.4 Grafo y Dijkstra 12
4.5 Operaciones compuestas del sistema 13
5. Justificación de la elección de estructuras 14
6. Cumplimiento de los requisitos del proyecto 15
7. Reportes para la toma de decisiones 16
8. Pruebas realizadas ..... **¡Error! Marcador no definido.**
9. Conclusiones 17
10. Anexos 18
Anexo A. Cómo compilar y ejecutar 18
Anexo B. Archivos del código fuente ..... **¡Error! Marcador no definido.**

1. Introducción

1.1 Objetivo

Este manual explica cómo diseñamos y armamos la aplicación para administrar una red de transporte urbano, hecha para el proyecto del Laboratorio 2 de Estructuras de Datos II.

Para este proyecto no usamos las colecciones que ya trae .NET, sino que programamos desde cero cuatro estructuras clave: una tabla hash, un grafo (con recorridos BFS, DFS y el algoritmo de Dijkstra), un sistema Union-Find y un trie.

El documento detalla por qué elegimos cada estructura según lo que más hace el programa (buscar, insertar, borrar, recorrer y conectar), cómo las implementamos y una comparativa de su rendimiento (en notación Big-O) frente a las alternativas oficiales de .NET.

1.2 Escenario elegido

Se eligió el escenario Red de transporte urbano. El sistema administra paradas y rutas de buses:

- Cada parada tiene un código único, un nombre y coordenadas (latitud y longitud).
- Cada ruta conecta dos paradas con un tiempo y un costo (quetzales), y puede operar en un solo sentido.
- El sistema permite registrar paradas y rutas, localizar una parada por su código, sugerir nombres mientras se escribe, calcular la ruta más rápida o más económica entre dos paradas y verificar si dos paradas pertenecen a la misma zona conectada antes de habilitar o calcular una ruta.

1.3 Datos y alcance

Los datos no están quemados en el código: se cargan desde archivos .csv o .txt o se ingresan desde la interfaz. Se incluyen datos de ejemplo de Quetzaltenango y municipios cercanos: 24 paradas y 33 rutas, organizadas en tres zonas desconectadas a propósito para poder demostrar el Union-Find.

1.4 Tecnologías utilizadas

Elemento	Tecnología	Uso
Lenguaje y plataforma	C# sobre .NET 8	Toda la lógica del programa
Interfaz gráfica	Windows Forms	Ventana principal, paneles, mapa y reportes
Mapa real	GMap.NET con teselas de OpenStreetMap	Fondo cartográfico (requiere internet; guarda caché en disco)
Mapa ligero	GDI+ (dibujo propio)	Respaldo sin internet con contorno de Guatemala incrustado
Archivos de datos	CSV / TXT	Carga y guardado de paradas y rutas

2. Arquitectura del sistema

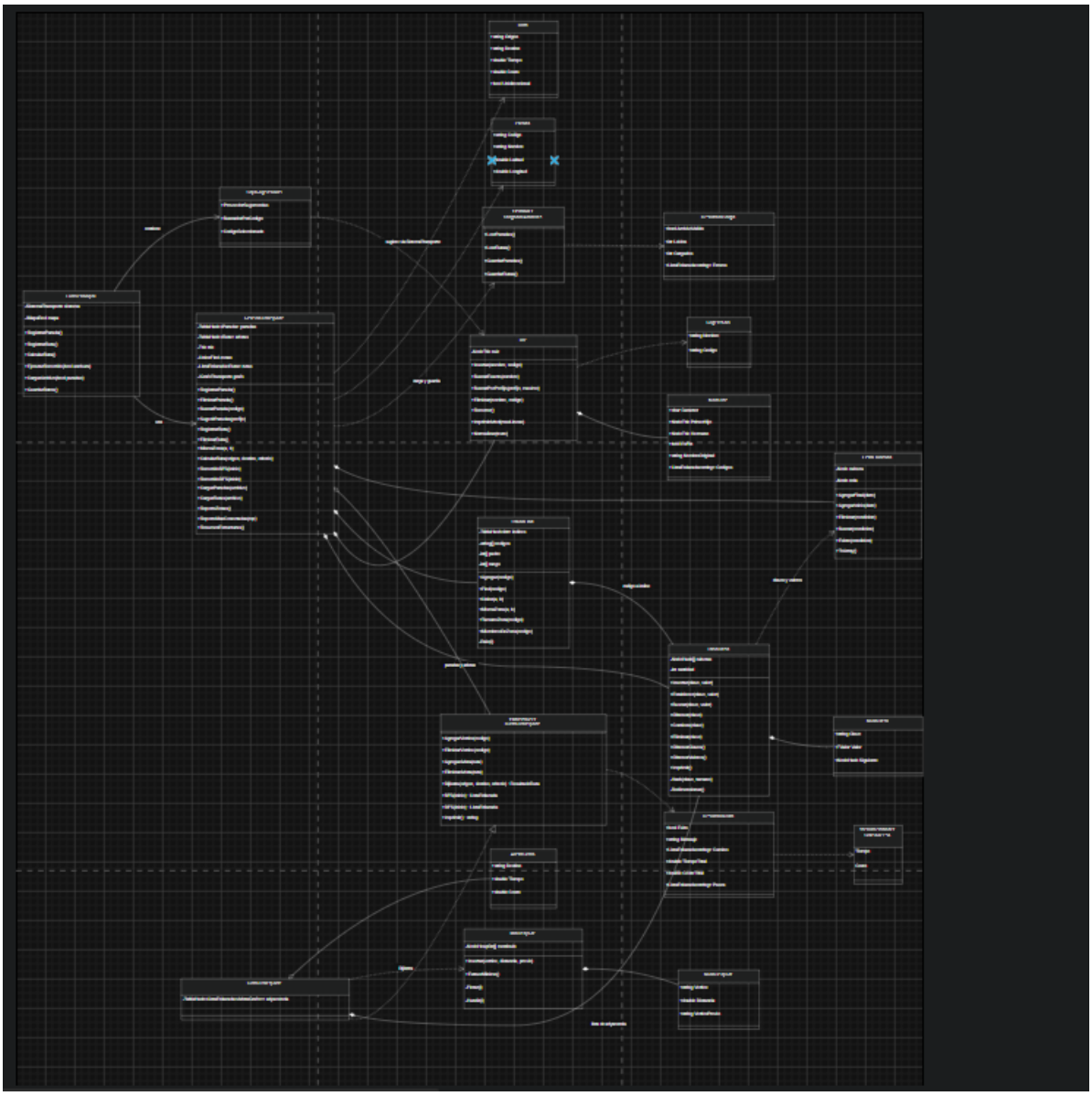
2.1 Organización del proyecto

Carpeta	Contenido	Responsabilidad
Models	`Parada`, `Ruta`, `ResultadoRuta`	Objetos de datos que viajan entre las capas
Estructuras	`ListaEnlazada`, `TablaHash`, `Trie`, `UnionFind`, `GrafoTransporte` (con `MinHeapStr`), `IGrafoTransporte`, `MapaBase`	Las estructuras de datos propias y su contrato con el resto del sistema
Servicios	`SistemaTransporte`, `CargadorArchivos`	Integración de las cuatro estructuras y lectura/escritura de archivos
UI	`FormPrincipal`, `MapaControl`, `MapaGMapControl`, `IMapaRed`, `FabricaMapa`, `CajaSugerencias`, `LineaTiempoControl`, `Tema`	Interfaz gráfica
Datos	`paradas.csv`, `rutas.csv`	Datos de ejemplo de Quetzaltenango

2.2 Responsables por estructura

Estructura / módulo	Archivo	Integrante responsable
Lista enlazada (soporte)	Estructuras/ListaEnlazada.cs	Integrante A
Tabla Hash	Estructuras/TablaHash.cs	Integrante A
Trie (autocompletado)	Estructuras/Trie.cs	Integrante A
Union-Find	Estructuras/UnionFind.cs	Integrante A
Carga de archivos	Servicios/CargadorArchivos.cs	Integrante A y Integrante B
Integración de estructuras	Servicios/SistemaTransporte.cs	Integrante A con revisión de B
Interfaz gráfica y mapa	UI/*.cs	Integrante A y Integrante B
Grafo: lista de adyacencia, BFS, DFS	Estructuras/GrafoTransporte.cs	Integrante B
Dijkstra y MinHeap	Estructuras/GrafoTransporte.cs (`MinHeapStr`)	Integrante B

2.3 Diagrama de clases



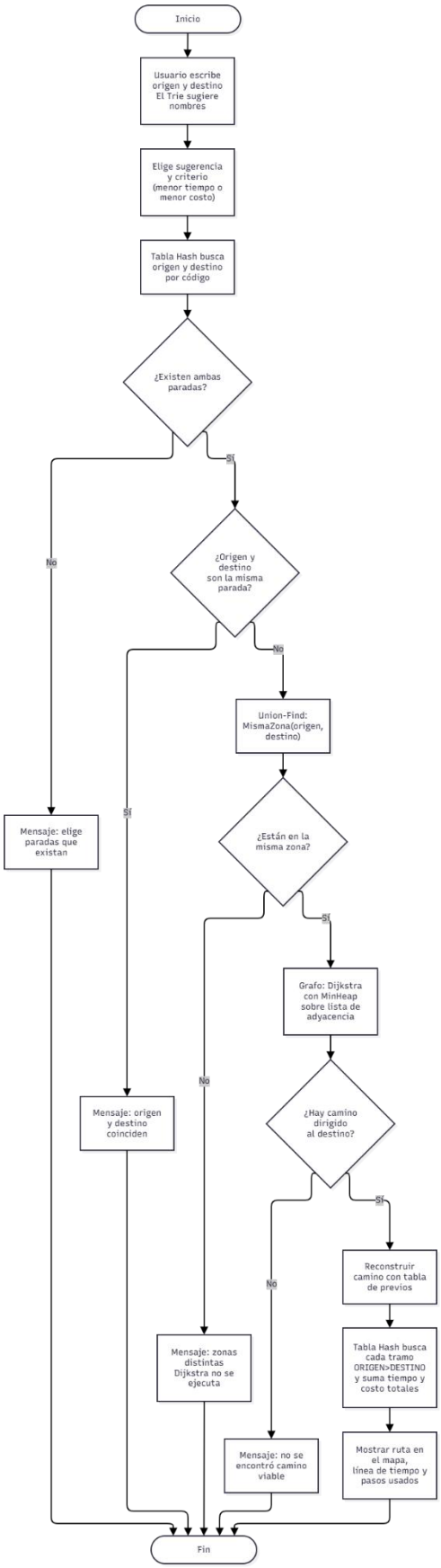
2.4 Integración de las estructuras

Las cuatro estructuras trabajan coordinadas dentro de un mismo flujo; ninguna queda aislada. La clase SistemaTransporte las orquesta:

Operación del usuario	Estructuras que participan (en orden)
Escribir en un buscador	Trie sugiere nombres con el prefijo escrito; el usuario elige una sugerencia y se obtiene su código.
Mostrar los datos de una parada	Tabla Hash localiza el registro completo por código en O(1).
Registrar una parada	Tabla Hash (guarda el registro), Trie (indexa el nombre) ,Union-Find (crea su zona) ,Grafo (crea el vértice).

Registrar una ruta	Tabla Hash (valida que existan ambas paradas) , Union-Find (verifica si las zonas son distintas) , Grafo (agrega la arista) ,Union-Find (une las zonas).
Calcular la ruta más rápida o económica	Tabla Hash (localiza origen y destino) , Union-Find (valida que estén en la misma zona) , Grafo (Dijkstra con MinHeap) , Tabla Hash (suma tiempo y costo de cada tramo).
Recorridos BFS y DFS	Tabla Hash (valida la parada de inicio) ,Grafo (recorrido) , resultado mostrado sobre el mapa.
Eliminar una ruta o una parada	Tabla Hash, Trie y Grafo eliminan el elemento; Union-Find se reconstruye porque no soporta quitar aristas.

2.5 Diagrama de flujo


Inicio
Usuario escribe origen y destino El Trie sugiere nombres
Elige sugerencia y criterio (menor tiempo o menor costo)
Tabla Hash busca origen y destino por código
¿Existen ambas paradas?
Sí
No
¿Origen y destino son la misma parada?
No
Sí
Mensaje: elige paradas que existan
Union-Find: MismaZona(origen, destino)
¿Están en la misma zona?
Sí
No
Mensaje: origen y destino coinciden
Grafo: Dijkstra con MinHeap sobre lista de adyacencia
¿Hay camino dirigido al destino?
Sí
No
Mensaje: zonas distintas Dijkstra no se ejecuta
Reconstruir camino con tabla de previos
Tabla Hash busca cada tramo ORIGEN>DESTINO y suma tiempo y costo totales
Mostrar ruta en el mapa, línea de tiempo y pasos usados
Mensaje: no se encontró camino viable
Fin

3. Implementación de las estructuras

3.1 Lista enlazada (soporte)

se implementó una lista enlazada simple con punteros a la cabeza y a la cola. Es la estructura auxiliar que usan todas las demás para devolver colecciones (listas de adyacencia, resultados de autocompletado, recorridos). Implementa IEnumerable<T> únicamente para poder usar foreach; todo su funcionamiento interno es propio.

- Agregar al final y al inicio: O(1).
- Buscar, eliminar por condición e indexador: O(n).

3.2 Tabla Hash

Descripción

Tabla Hash genérica con claves de tipo texto y encadenamiento separado: cada cubeta es una lista enlazada de nodos. Cuando el factor de carga supera 0.75 se pasa a un tamaño primo mayor y se reubican los nodos.

Operaciones

- Insertar: rechaza claves repetidas, aplica rehashing si hace falta e inserta al inicio de la cadena.
- Buscar, Obtener, Contiene: calculan la cubeta y recorren su cadena.
- Eliminar: enlaza el nodo anterior con el siguiente.
- Establecer: inserta o reemplaza el valor de una clave.
- ObtenerValores, ObtenerClaves e Imprimir: recorren todas las cubetas. Imprimir muestra las cubetas ocupadas y sus colisiones.

Dónde se usa

- Paradas por código (TablaHash<Parada>): localizar el registro completo en O(1).
- Tramos por par origen-destino (TablaHash<Ruta>, clave ORIGEN>DESTINO): obtener el tiempo y costo de un tramo y evitar rutas duplicadas.
- Listas de adyacencia del grafo, tablas de distancias y visitados de Dijkstra/BFS/DFS, y el índice de códigos de Union-Find.

Justificación

La operación más frecuente sobre las paradas es buscar una por su código exacto. La tabla hash la resuelve en O(1) promedio, frente a O(n) de una lista o O(log n) de un árbol balanceado. No necesita orden entre claves, por lo que no hace falta pagar el costo de mantenerlo.

3.3 Trie

Descripción

cada nodo representa un carácter y cada camino desde la raíz forma un nombre de parada. Los hijos de un nodo se guardan en una lista enlazada ordenada por carácter (representación "primer hijo / hermano"), por lo que no se usa un arreglo de 256 posiciones por nodo ni un Dictionary, y el recorrido sale en orden alfabético.

Normalización

Antes de insertar o buscar, el texto se pasa a minúsculas, se eliminan tildes y diéresis y se colapsan los espacios. Así "Cívico", "civico" y "CÍVICO" coinciden. El nodo final conserva el nombre original y la lista de códigos, porque dos paradas pueden compartir nombre.

Operaciones

- Insertar(nombre, codigo): crea los nodos faltantes y marca el fin de palabra.
- BuscarExacto: devuelve los códigos asociados a un nombre.
- BuscarPorPrefijo(prefijo, maximo): desciende hasta el nodo del prefijo y recorre su subárbol en orden alfabético hasta reunir el máximo de sugerencias. Es la base del autocompletado.
- Eliminar(nombre, codigo): quita el código y poda hacia arriba los nodos que quedaron sin hijos ni palabra, para no dejar basura en el árbol.
- Recorrer e ImprimirArbol: listan todos los nombres y dibujan el árbol indentado.

Justificación

El autocompletado necesita todas las paradas que empiezan con un prefijo. En el Trie el costo depende del largo del prefijo y de los resultados devueltos, no de cuántas paradas existan. Una tabla hash no sirve para esto (solo encuentra claves exactas) y una lista con StartsWith recorre todas las paradas en cada pulsación de tecla.

3.4 Union-Find (conjuntos disjuntos)

Descripción

Cada zona conectada de la red es un conjunto. Se implementa con arreglos de padres y rangos; los códigos de parada se traducen a índices con una Tabla Hash propia.

- Agregar (MakeSet): crea una zona nueva con un elemento.
- Find: devuelve el representante de la zona aplicando **compresión de camino**: cada nodo visitado queda apuntando directo a la raíz.
- Union: une dos zonas por **rango** (el árbol más bajo se cuelga del más alto) para mantener los árboles planos.
- MismaZona, TamanoZona, MiembrosDeZona: consultas sobre la conectividad.

Decisiones y limitaciones

- Union-Find no soporta eliminar aristas. Cuando se elimina una ruta o una parada, SistemaTransporte reconstruye la estructura recorriendo paradas y rutas.
- Las rutas de un solo sentido cuentan como conexión "débil": dos paradas pueden estar en la misma zona y aun así no tener un camino dirigido entre ellas. En ese caso Dijkstra informa que no existe ruta.
- Las zonas se numeran 1, 2, 3... ordenadas por el menor código de parada que contienen, para que la numeración sea estable.

Justificación

Antes de ejecutar Dijkstra se necesita saber si dos paradas están conectadas. Con Union-Find la respuesta cuesta casi O(1); con un BFS o DFS costaría O(V + E) en cada consulta, y cuando la respuesta es "no" habría que explorar toda la zona para descubrirlo.

3.5 Grafo, recorridos y Dijkstra

Representación

Grafo **dirigido y ponderado** con **lista de adyacencia**: una TablaHash<ListaEnlazada<AristaGrafo>> donde la clave es el código de la parada y el valor es la lista de sus conexiones salientes. Cada AristaGrafo guarda el destino, el tiempo y el costo. Una ruta bidireccional agrega dos aristas (una por sentido); una unidireccional agrega solo la de origen a destino.

Operaciones

- AgregarVertice, AgregarArista, EliminarArista, EliminarVertice (quita el vértice y las aristas que llegan a él).
- BFS(inicio): recorrido en anchura; usa una Queue<string> únicamente como estructura de control del orden de visita, como permite el enunciado, y una tabla hash de visitados.
- DFS(inicio): recorrido en profundidad con recursión y tabla hash de visitados.
- Dijkstra(origen, destino, criterio): camino de menor tiempo o de menor costo.
- Imprimir: muestra la lista de adyacencia con tiempo y costo de cada arista.

Dijkstra con MinHeap propio

Se implementó un montículo binario mínimo (MinHeapStr) sobre un arreglo, con Insertar y ExtraerMinimo en O(log n) mediante las operaciones de flotar y hundir. Dijkstra mantiene tres tablas hash (distancias, vértice previo y procesados). Cuando se encuentra una distancia mejor, se inserta una nueva entrada en el montículo sin modificar la anterior; al extraerla, si el vértice ya fue procesado se descarta ("eliminación perezosa"). Al terminar se reconstruye el camino siguiendo los vértices previos desde el destino.

El criterio de peso (tiempo o costo) se elige al calcular, por lo que la misma estructura responde "la ruta más rápida" y "la más económica". Dijkstra requiere pesos no negativos, condición que SistemaTransporte garantiza al validar que el tiempo sea mayor que cero y el costo no sea negativo.

Justificación

La red es **dispersa**: cada parada se conecta con pocas otras. La lista de adyacencia ocupa O(V + E) de memoria, mientras que una matriz ocuparía O(V^2). Dijkstra con montículo evita el barrido lineal de O(V^2) para encontrar el siguiente vértice.

3.6 Carga y guardado de archivos

CargadorArchivos lee paradas y rutas desde .csv o .txt. Antes de leer verifica que **la ruta exista** y que la **extensión sea válida**. Detecta el separador (coma, punto y coma o tabulación), ignora el encabezado, las líneas vacías y las que empiezan con #, y reporta cada fila con error (columnas incompletas, números inválidos, códigos repetidos, paradas inexistentes) **sin detener la carga**. Los errores se muestran en la pestaña Análisis. También puede guardar lo que hay en memoria en archivos CSV.

Archivo	Formato de cada línea	Ejemplo
paradas	codigo,nombre,latitud,longitud	X01,Parque Centro América,14.8347,-91.5181
rutas	origen,destino,tiempo,costo,unidireccional	X01,X02,3,3.50,0

3.7 Interfaz gráfica y mapa

La interfaz es una aplicación de Windows Forms con un panel lateral de cuatro pestañas (Paradas, Rutas, Calcular y Análisis), un mapa central y una franja inferior con el recorrido paso a paso. El mapa puede ser **real** (OpenStreetMap mediante GMap.NET, con caché en disco para ver sin conexión lo ya visitado) o **ligero** (dibujo propio con el contorno de Guatemala incrustado, que funciona sin internet). Ambos implementan la interfaz IMapaRed, por lo que se puede cambiar de uno a otro con un botón sin tocar el resto del programa.

4. Análisis de complejidad (Big-O)

4.1 Tabla Hash

Operación	Propia (promedio)	Peor caso	Alternativa nativa: Dictionary<K,V>
Insertar	O(1) amortizado	O(n)	O(1) amortizado
Buscar / Contiene	O(1)	O(n)	O(1)
Eliminar	O(1)	O(n)	O(1)
Recorrer / Imprimir	O(m + n)	O(m + n)	O(n)
Rehashing	O(m + n), espaciado	—	Redimensiona de forma equivalente

4.2 Trie

Operación	Propia	Alternativa nativa
Insertar	O(L · s)	List<string>.Add: O(1) amortizado, pero sin soporte de prefijos
Buscar nombre exacto	O(L · s)	Dictionary: O(1); List.Contains: O(n · L)
Búsqueda por prefijo (autocompletado)	O(L · s + t), t = nodos visitados para reunir k resultados	List<string> con StartsWith: O(n · L); SortedSet con GetViewBetween: O(log n) + k, aproximadamente
Eliminar	O(L · s)	List.Remove: O(n · L)
Recorrer en orden alfabético	O(N), N = nodos	SortedSet: O(n); List: requiere ordenar, O(n log n)

4.3 Union-Find

Operación	Propia	Alternativa nativa
Agregar (MakeSet)	O(1) amortizado	HashSet.Add: O(1)
Find / MismaZona	O(α(n)) amortizado	BFS o DFS por consulta: O(V + E)
Union	O(α(n)) amortizado	Fusionar HashSets: O(tamaño del menor)
Tamaño o miembros de una zona	O(n · α(n))	Dictionary de listas: O(tamaño de la zona)
Reconstruir tras eliminar	O(n + E · α(n))	Recalcular componentes con BFS: O(V + E)

4.4 Grafo y Dijkstra

Operación	Propia	Alternativa nativa / comparación
Agregar vértice	O(1) promedio	Dictionary<string, List<Arista>>: O(1)
Agregar arista	O(1) promedio	List.Add: O(1) amortizado
Eliminar arista	O(d)	List.Remove: O(d)
Eliminar vértice	O(V + E)	Igual: hay que quitar las aristas entrantes
BFS	O(V + E)	Igual con Queue<T>
DFS	O(V + E)	Igual; la recursión puede llegar a profundidad V
Dijkstra con MinHeap propio	O((V + E) log V)	PriorityQueue<TElement, TPriority>: igual orden. Sin montículo, $O(V^2)$.

Memoria	O(V + E) con lista de adyacencia	Matriz de adyacencia: $O(V^2)$

4.5 Operaciones compuestas del sistema

Operación	Complejidad	Comentario
Registrar parada	$O(L \cdot s)$	Dominado por la inserción en el Trie; Hash, Union-Find y Grafo son O(1)
Registrar ruta	O(1) promedio	Búsquedas en hash, agregar arista y Union en $O(\alpha(n))$
Sugerir paradas	$O(L \cdot s + t)$	Trie
Calcular ruta	$O((V + E) \log V)$	Dijkstra domina; hash, Union-Find y suma de tramos son menores
Eliminar ruta o parada	O(V + E)	Se reconstruye el Union-Find
Listar paradas ordenadas	$O(n^2)$	Ordenamiento por inserción; aceptable para cientos de paradas. Una mejora sería un ordenamiento O(n log n).

5. Justificación de la elección de estructuras

Operación frecuente	Estructura elegida	Por qué	Alternativa descartada
Buscar una parada por código	Tabla Hash	Acceso directo en O(1) promedio por clave exacta	Lista: O(n). Árbol balanceado: O(log n) y mantiene un orden innecesario.
Autocompletar un nombre por prefijo	Trie	El costo depende del prefijo, no de la cantidad de paradas; devuelve sugerencias en orden alfabético	Hash: no soporta prefijos. Lista con StartsWith: O(n · L) en cada tecla.
Representar rutas y calcular caminos	Grafo con lista de adyacencia	Red dispersa: memoria O(V + E); recorre vecinos en O(d); admite aristas dirigidas y pesos	Matriz de adyacencia: $O(V^2)$ de memoria para pocas conexiones.
Ruta más rápida o económica	Dijkstra con MinHeap	Pesos no negativos; O((V + E) log V)	BFS: ignora los pesos. Dijkstra sin montículo: $O(V^2)$.
¿Hay conexión entre dos paradas?	Union-Find	Responde en casi O(1) y evita explorar el grafo cuando no hay camino	BFS o DFS por consulta: O(V + E).
Visitar paradas en orden por niveles	Cola (Queue<T>)	Estructura de control permitida por el enunciado para BFS	—

6. Cumplimiento de los requisitos del proyecto

Requisito	Cómo se cumple
1. Estructuras obligatorias	Tabla Hash, Grafo, Union-Find y Trie implementadas y usadas; responsables indicados en la sección 2.2 y en los comentarios del código.
2. Integración entre estructuras	SistemaTransporte coordina las cuatro en un mismo flujo (sección 2.4).
3. Operaciones	Inserción, búsqueda, eliminación, recorrido e impresión en cada estructura; BFS, DFS y Dijkstra en el grafo; Union y Find con compresión de camino; búsqueda por prefijo en el Trie.
4. Análisis de complejidad	Sección 4, con comparación frente a las alternativas nativas de .NET.
5. Datos dinámicos	Carga desde .csv / .txt y registro manual desde la interfaz; ningún dato está quemado en el código.
6. Menú interactivo	Interfaz gráfica con pestañas que permiten ejecutar todas las operaciones de las cuatro estructuras.
7. Implementación propia	Ninguna estructura usa colecciones nativas para su lógica. Ver la nota siguiente.
Interfaz gráfica obligatoria	Aplicación Windows Forms con mapa.

7. Reportes para la toma de decisiones

El sistema organiza la información de la red y genera reportes que ayudan a decidir, desde la pestaña Análisis:

- Zonas conectadas: cuántas zonas tiene la red, cuántas paradas hay en cada una y qué paradas las forman. Identifica sectores aislados a los que convendría conectar con una ruta nueva.
- Paradas más conectadas: ranking de paradas por cantidad de rutas; señala los puntos de transbordo más importantes.
- Ruta más rápida o más económica: comparación entre criterios, con tiempo total, costo total y paradas intermedias.
- Estado de las estructuras: elementos, cubetas, factor de carga y colisiones de las tablas hash; nodos del Trie; zonas del Union-Find.

8. Conclusiones

Redacta aquí las conclusiones del grupo con tus propias palabras. Algunas ideas, en función de lo que se vio en el proyecto:

- Cada estructura resolvió la operación para la que se eligió, y la complejidad medida coincide con el análisis teórico.
- La integración (Trie, Tabla Hash, Union-Find y Grafo en un mismo flujo) evita trabajo innecesario: Union-Find descarta rápido los casos sin conexión antes de ejecutar Dijkstra.
- Definir un contrato común (IGrafoTransporte) permitió trabajar en paralelo y probar la interfaz antes de tener el grafo terminado.
- Limitaciones: Union-Find no soporta eliminar aristas (se reconstruye), los datos de ejemplo tienen coordenadas estimadas y las rutas se dibujan como líneas rectas entre paradas.

9. Anexos

Anexo. Cómo compilar y ejecutar

1. Instalar el SDK de .NET 8 y abrir la carpeta del proyecto (la que contiene `TransporteUrbano.csproj`) en Visual Studio Code.
2. Con conexión a internet la primera vez, abrir una terminal y ejecutar `dotnet run` (se descarga el paquete GMap.NET.WinForms automáticamente).
3. Para abrir con el mapa que no necesita internet: `dotnet run -- --ligero`.